Tutorial de Git - Capítulo 2

1. Obtener un Repositorio Git

Existen dos formas principales de comenzar con un proyecto en Git:

- **Inicializar en un directorio existente:** Navegar a la carpeta del proyecto y ejecutar:

git init

Esto crea el subdirectorio oculto .git con la estructura base del repositorio.

- **Clonar un repositorio existente:** Obtiene una copia completa de todos los datos y el historial del servidor remoto:

git clone https://github.com/libgit2/libgit2

Para asignarle un nombre personalizado a la carpeta de destino:

git clone https://github.com/libgit2/libgit2 mi_carpeta

2. Registrar Cambios en el Repositorio

- **Ciclo de vida de los archivos:** Los archivos se dividen en **rastreados** (tracked, conocidos en el último snapshot) y **no rastreados** (untracked).
- **Verificar el estado del proyecto:**

git status

Muestra en qué rama te encuentras y qué archivos han sido modificados, preparados o no rastreados.

- **Rastrear y preparar archivos (Staging):** Para comenzar a rastrear un archivo nuevo o preparar cambios de uno modificado:

git add README.md

El comando git add añade el contenido exacto del archivo al área de preparación para la siguiente confirmación.

- **Ignorar archivos no deseados (.gitignore):** Se crea un archivo nombrado .gitignore para listar patrones de nombres o extensiones que Git no debe rastrear (archivos de registro, compilados, temporales). Ejemplo de archivo .gitignore:

Ignorar todos los archivos .a

*.a

Pero rastrear lib.a

!lib.a

Ignorar la carpeta build/

build/

- **Inspeccionar cambios preparados y no preparados:**
    - Cambios no preparados en el directorio de trabajo: git diff
- Cambios preparados que irán en el próximo commit: git diff --staged
- **Confirmar cambios (Committing):** Guarda la instantánea de los archivos en el área de preparación.

git commit -m "Mensaje descriptivo del cambio"

Para preparar automáticamente todos los archivos rastreados modificados y confirmarlos en un solo paso:

git commit -a -m "Mensaje del commit"

3. Ver el Historial de Commits

- **Comando básico:** git log

Lista los commits realizados en orden cronológico inverso con su SHA-1, autor, fecha y mensaje.

- **Opciones útiles de formato:**
    - Ver las diferencias introducidas en cada commit: git log -p.
    - Resumen de estadísticas de archivos modificados: git log --stat.
    - Formato condensado en una línea: git log --pretty=oneline.
    - Visualización de ramas e historial de fusiones en gráfico ASCII:

git log --pretty=format:"%h %s" --graph

4. Deshacer Cosas

- **Modificar el último commit (Amendar):** Si olvidaste añadir un archivo o redactaste mal el mensaje del commit recién hecho:

git commit --amend

- **Retirar un archivo del Área de Preparación (Unstage):** Para quitar un archivo preparado sin perder sus cambios en el directorio de trabajo:

git reset HEAD <archivo>

En versiones recientes de Git:

git restore --staged <archivo>

- **Deshacer modificaciones de un archivo local:** Para descartar los cambios locales y volver a la versión del último commit:

git checkout -- <archivo>

En versiones recientes de Git:

git restore <archivo>

5. Trabajar con Remotos

- **Listar remotos configurados:** git remote -v
- **Añadir un repositorio remoto:** git remote add pb https://github.com/paulboone/ticgit
- **Obtener cambios del remoto (Fetch y Pull):**
    - Descarga los datos del remoto sin fusionar automáticamente en tus ramas:

git fetch <remoto>

- Descarga y fusiona automáticamente la rama remota en la rama actual:

git pull.

- **Enviar cambios al servidor (Push):** git push origin master

6. Etiquetado (Tagging)

Permite marcar puntos específicos en la historia como importantes (por ejemplo, versiones de software v1.0).

- **Listar etiquetas:** git tag.
- **Etiquetas Anotadas:** Almacenan información del etiquetador, fecha y mensaje:

git tag -a v1.4 -m "Versión 1.4 de producción"

- **Etiquetas Ligeras:** Un simple puntero a un commit específico sin información adicional:

git tag v1.4-lw

- **Etiquetar commits pasados:** git tag -a v1.2 9fceb02
- **Enviar etiquetas al remoto:** git push origin v1.4

Para enviar todas las etiquetas:

git push origin --tags

7. Alias de Git

Permite crear accesos directos personalizados para comandos complejos o de uso frecuente:

git config --global alias.co checkout

git config --global alias.br branch

git config --global alias.ci commit

git config --global alias.st status

git config --global alias.unstage 'reset HEAD --'